XRADIO Efuse Tool User Guide

Revision 2.2

Aug 08, 2020

Declaration

THIS DOCUMENTATION IS THE ORIGINAL WORK AND COPYRIGHTED PROPERTY OF XRADIO TECHNOLOGY ("XRADIO"). REPRODUCTION IN WHOLE OR IN PART MUST OBTAIN THE WRITTEN APPROVAL OF XRADIO AND GIVE CLEAR ACKNOWLEDGEMENT TO THE COPYRIGHT OWNER.

THE PURCHASED PRODUCTS, SERVICES AND FEATURES ARE STIPULATED BY THE CONTRACT MADE BETWEEN XRADIO AND THE CUSTOMER. PLEASE READ THE TERMS AND CONDITIONS OF THE CONTRACT AND RELEVANT INSTRUCTIONS CAREFULLY BEFORE USING, AND FOLLOW THE INSTRUCTIONS IN THIS DOCUMENTATION STRICTLY. XRADIO ASSUMES NO RESPONSIBILITY FOR THE CONSEQUENCES OF IMPROPER USE (INCLUDING BUT NOT LIMITED TO OVERVOLTAGE, OVERCLOCK, OR EXCESSIVE TEMPERATURE).

THE INFORMATION FURNISHED BY XRADIO IS PROVIDED JUST AS A REFERENCE OR TYPICAL APPLICATIONS, ALL STATEMENTS, INFORMATION, AND RECOMMENDATIONS IN THIS DOCUMENT DO NOT CONSTITUTE A WARRANTY OF ANY KIND, EXPRESS OR IMPLIED. XRADIO RESERVES THE RIGHT TO MAKE CHANGES IN CIRCUIT DESIGN AND/OR SPECIFICATIONS AT ANY TIME WITHOUT NOTICE.

NOR FOR ANY INFRINGEMENTS OF PATENTS OR OTHER RIGHTS OF THE THIRD PARTIES WHICH MAY RESULT FROM ITS USE. NO LICENSE IS GRANTED BY IMPLICATION OR OTHERWISE UNDER ANY PATENT OR PATENT RIGHTS OF XRADIO.THIRD PARTY LICENCES MAY BE REQUIRED TO IMPLEMENT THE SOLUTION/PRODUCT. CUSTOMERS SHALL BE SOLELY RESPONSIBLE TO OBTAIN ALL APPROPRIATELY REQUIRED THIRD PARTY LICENCES. XRADIO SHALL NOT BE LIABLE FOR ANY LICENCE FEE OR ROYALTY DUE IN RESPECT OF ANY REQUIRED THIRD PARTY LICENCE. XRADIO SHALL HAVE NO WARRANTY, INDEMNITY OR OTHER OBLIGATIONS WITH RESPECT TO MATTERS COVERED UNDER ANY REQUIRED THIRD PARTY LICENCE.

Revision History

Version	Date	Summary of Changes
1.0	2017-11-23	Initial Version
2.0	2019-08-07	Add XR872 version
2.1	2019-10-11	Fix file name
2.2	2020-08-08	Format Optimize

Table 1- 1 Revision History

Contents

Tables

Figures

1 Overview

1.1 Introduction to EFUSE API

EFUSE is used to program the eFuse of the XR872 chip and transmit and receive command responses via the UART serial port. efuse_api.dll encapsulates a series of interface functions for these operations. The corresponding API description can be found in the efuse_api.h header file.

1.2 API Configuration Method

The following takes the sample project as an example to briefly introduce the configuration method of efuse_api. There are three files in total that need to be configured: efuse_api.dll, efuse_api.lib, and efuse_api.h.

1.2.1 efuse_api.dll

This file is the dynamic link library of the API. You only need to put it and the exe application that calls it in the same directory.

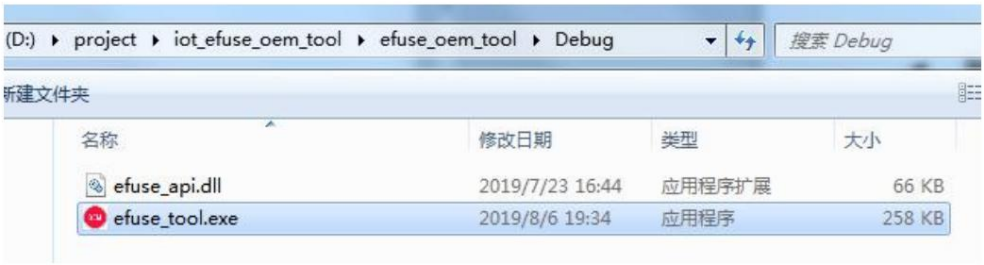

Figure 1-1 Dynamic link library dll file

1.2.2 efuse_api.lib

This file is the import library for the efuse_api.dll dynamic link library, not a static link library file. You need to place this file in the project directory and add the following statement to the file that uses the API to import the library file.

```
#pragma comment(lib,"efuse_api.lib")
```

As shown in the figure:

```
//

#pragma once

#include "afxwin.h"
#include "efuse_api.h"
#include "Comm.h"
#include "dlgEfuse709.h"
#include "dlgEfuse871.h"
#include "dlgEfuse872.h"
#include "afxcmn.h"
#pragma comment(lib,"efuse_api.lib")

const CString strVersion = (_T("Efuse tool version: 2.0.08061"));
const CString strCopyright = (_T("Copyright (C): XRadio Technology"));
```

Figure 1-2 Code modification to add import library lib file

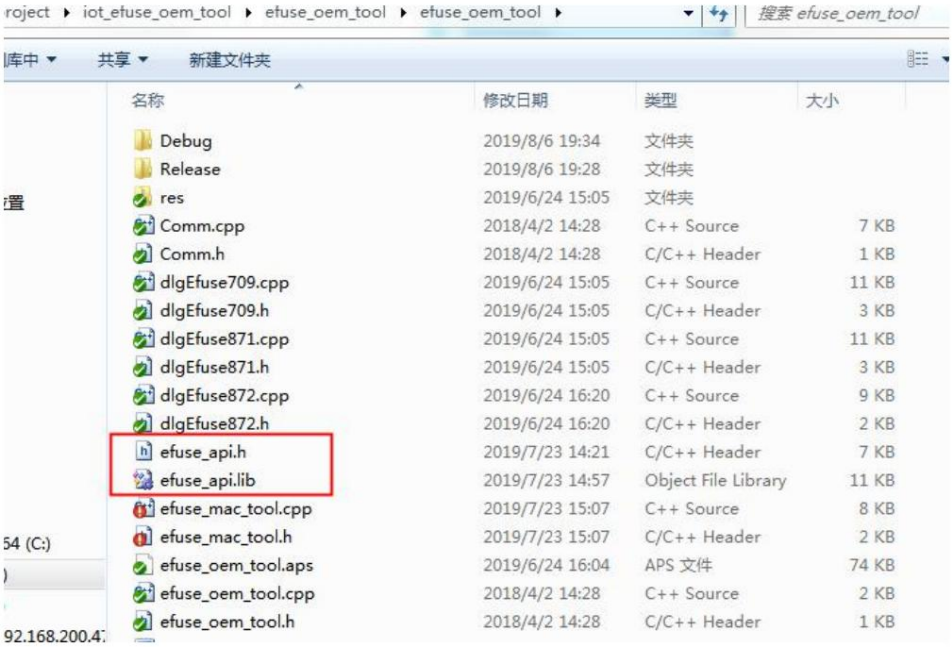

Figure 1-3 Placing the lib file in the project

1.2.3 efuse_api.h

This file is the API header file, which contains a brief introduction to the API. You need to place this file in the project directory and include it in the files that use the API, as shown in Figures 1-2 and 1-3.

2 API Introduction

2.1 EFUSE_GetMsg

Prototype: char* EFUSE_GetMsg()

Description: Get the status information of the last command execution.

Input parameters: None

Return value: pointer to status information string

Example:

```
m_strMsg.Format(_T("%s"), EFUSE_GetMsg());

GetDlgItem(IDC_STATUS)->SetWindowText(m_strMsg);
```

2.2 EFUSE_SetHashKey

ÿÿÿint EFUSE_SetHashKey(char* buf, int len)

Description: Sets the hash key.

Input Parameters: buf - pointer to hash string

len - the length of the hash string, maximum 256

Return value: status value, 0 - success, non-0 - failure

Example:

```
int return = 0;

CString strKey = _T(“12345678”);

if (EFUSE_SetHashKey(m_strKey.GetBuffer(), m_strKey.GetLength()))

        right = -1;

m_strKey.ReleaseBuffer();

Return ret;
```

2.3 EFUSE_InitEfuse

ÿÿÿint EFUSE_InitEfuse(int comNum, char* keyBuf, int len)

Description: Initialize EFUSE, including serial port and key data, and enter EFUSE mode.

Input parameter: comNum - serial port number

keyBuf - pointer to the hash string

len - the length of the hash string, maximum 256

Return value: status value, 0 - success, non-0 - failure

Example:

```
int com, keyLen;
com = pMain->m_ComNum.GetCurSel();
keyLen = pMain->m_strKey.GetLength();
if (EFUSE_InitEfuse(com, pMain->m_strKey.GetBuffer(), keyLen))
{
        pMain->m_strMsg.Format(_T("%s"), EFUSE_GetMsg());
        SendMessage(pMain->m_hWnd, EFUSE_MSG, NULL, NULL);
        EFUSE_ExitEfuse();
        return -1;
}
```

2.4 EFUSE_ExitEfuse

Prototype: int EFUSE_ExitEfuse()

Description: Exit EFUSE mode and close the serial port.

Input parameters: None

Return value: status value, 0 - success, non-0 - failure

Example:

```
int com, keyLen;

com = pMain->m_ComNum.GetCurSel();

keyLen = pMain->m_strKey.GetLength();

if (EFUSE_InitEfuse(com, pMain->m_strKey.GetBuffer(), keyLen))

{

        pMain->m_strMsg.Format(_T("%s"), EFUSE_GetMsg());

        SendMessage(pMain->m_hWnd, EFUSE_MSG, NULL, NULL);

        EFUSE_ExitEfuse();

        return -1;

}

pMain->m_strKey.ReleaseBuffer();
```

2.5 EFUSE_WriteHoscType

ÿÿÿint EFUSE_WriteHoscType(int hosc)

Description: Write HOSC type.

Input parameters: hosc: 0 - 26M 1 - 40M 2 - 24M 3 - 52M -1 - invalid

Return value: status value, 0 - success, non-0 - failure

Example:

```
        if (EFUSE_WriteHoscType(pMain->m_iHoscVal))

        {

                pMain->m_strMsg.Format(_T("%s"), EFUSE_GetMsg());

                SendMessage(pMain->m_hWnd, EFUSE_MSG, NULL, NULL);

                EFUSE_ExitEfuse();

                return -1;

        }
```

2.6 EFUSE_ReadHoscType

ÿÿÿint EFUSE_ReadHoscType(int* hosc)

Description: Read HOSC type.

Input parameters: hosc: 0 - 26M 1 - 40M 2 - 24M 3 - 52M -1 - invalid

Return value: status value, 0 - success, non-0 - failure

Example:

```
if (EFUSE_ReadHoscType(&(pMain->m_iHoscVal)))
{
    pMain->m_strMsg.Format(_T("%s"), EFUSE_GetMsg());
    SendMessage(pMain->m_hWnd, EFUSE_MSG, NULL, NULL);
    EFUSE_ExitEfuse();
    return -1;
}
```

2.7 EFUSE_WriteScrBoot

ÿÿÿint EFUSE_WriteScrBoot(char* hash, int len)

Description: Write the hash value of secure boot.

Input parameter: hash - array used to store hash string, only 32 bytes

len - the length of the hash string, can only be 32

Return value: status value, 0 - success, non-0 - failure

Example:

```
if (EFUSE_WriteScrBoot(pMain->m_strScrBootVal.GetBuffer(), 32))
{
    pMain->m_strMsg.Format(_T("%s"), EFUSE_GetMsg());
    SendMessage(pMain->m_hWnd, EFUSE_MSG, NULL, NULL);
    EFUSE_ExitEfuse();
    return -1;
}
pMain->m_strScrBootVal.ReleaseBuffer();
```

2.8 EFUSE_ReadScrBoot

ÿÿÿint EFUSE_ReadScrBoot(char* hash, int len)

Description: Read the hash value of secure boot.

Input parameters: hash - array to store hashed strings

len - the length of the array storing the hash string, which must be greater than or equal to 32

Return value: status value, 0 - success, non-0 - failure

Example:

```
char buf[33];
if (EFUSE_ReadScrBoot(buf, sizeof(buf)))
{
    pMain->m_strMsg.Format(_T("%s"), EFUSE_GetMsg());
    SendMessage(pMain->m_hWnd, EFUSE_MSG, NULL, NULL);
    EFUSE_ExitEfuse();
    return -1;
}
buf[32] = '\0';
pMain->m_strScrBootVal.Format(_T("%s"), buf);
```

2.9 EFUSE_WriteDcxoTrim

ÿÿÿint EFUSE_WriteDcxoTrim(char value)

Description: Write DCXO TRIM value.

Input parameter: value - DCXO TRIM value

Return value: status value, 0 - success, non-0 - failure

Example:

```
BYTE dcxoTrim;
dcxoTrim = (BYTE)_tcstoul(pMain->m_strDcxoTrimVal, NULL, 16);
if (EFUSE_WriteDcxoTrim(dcxoTrim))
{
    pMain->m_strMsg.Format(_T("%s"), EFUSE_GetMsg());
    SendMessage(pMain->m_hWnd, EFUSE_MSG, NULL, NULL);
    EFUSE_ExitEfuse();
    return -1;
}
```

2.10 EFUSE_ReadDcxoTrim

Prototype: int EFUSE_ReadDcxoTrim(char* value)

Description: Read DCXO TRIM value.

Input parameter: value - pointer to store DCXO TRIM value

Return value: status value, 0 - success, non-0 - failure

Example:

```
char dcxoTrim;
if (EFUSE_ReadDcxoTrim(&dcxoTrim))
{
    pMain->m_strMsg.Format(_T("%s"), EFUSE_GetMsg());
    SendMessage(pMain->m_hWnd, EFUSE_MSG, NULL, NULL);
    EFUSE_ExitEfuse();
    return -1;
}
pMain->m_strDcxoTrimVal.Format(_T("%02X"), dcxoTrim);
```

2.11 EFUSE_WritePoutCal

ÿÿÿint EFUSE_WritePoutCal(char* rfCal, int len)

Description: Write the POUT CAL value.

Input parameter: rfCal - array of POUT CAL values, only 3 bytes, only the lower 7 bits of each byte are valid

len - the length of the POUT CAL value array, can only be 3

Return value: status value, 0 - success, non-0 - failure

Example:

```
char poutCal[3];
poutCal[0] = (BYTE)_tcstoul(pMain->m_strPoutCal1, NULL, 16);
poutCal[1] = (BYTE)_tcstoul(pMain->m_strPoutCal2, NULL, 16);
poutCal[2] = (BYTE)_tcstoul(pMain->m_strPoutCal3, NULL, 16);
if (EFUSE_WritePoutCal(poutCal, 3))
{
    pMain->m_strMsg.Format(_T("%s"), EFUSE_GetMsg());
    SendMessage(pMain->m_hWnd, EFUSE_MSG, NULL, NULL);
    EFUSE_ExitEfuse();
    return -1;
}
```

2.12 EFUSE_ReadPoutCal

ÿÿÿint EFUSE_ReadPoutCal(char* rfCal, int len)

Description: Read the POUT CAL value.

Input parameter: rfCal - array to store POUT CAL values

len - the length of the array storing the POUT CAL value, greater than or equal to 3

Return value: status value, 0 - success, non-0 - failure

Example:

```
char poutCal[3];
if (EFUSE_ReadPoutCal(poutCal, sizeof(poutCal)))
{
    pMain->m_strMsg.Format(_T("%s"), EFUSE_GetMsg());
    SendMessage(pMain->m_hWnd, EFUSE_MSG, NULL, NULL);
    EFUSE_ExitEfuse();
    return -1;
}
pMain->m_strPoutCal1.Format(_T("%02X"), poutCal[0]);
```

2.13 EFUSE_WriteMacAddr

ÿÿÿint EFUSE_WriteMacAddr(char* mac, int len)

Description: Write the MAC address.

Input parameter: mac - pointer to MAC address array, only 6 bytes

len - the length of the MAC address array, can only be 6

Return value: status value, 0 - success, non-0 - failure

Example:

```
char mac[6];
int i;
CString strLeft, strRight;
strRight = pMain->m_strMacAddr;
for (i = 0;i < 6;i++)
{
    strLeft = strRight.Left(2);
    strRight = strRight.Right(strRight.GetLength() - 2);
    mac[i] = (BYTE)_tcstoul(strLeft, NULL, 16);
}
if (EFUSE_WriteMacAddr(mac, 6))
{
    pMain->m_strMsg.Format(_T("%s"), EFUSE_GetMsg());
    SendMessage(pMain->m_hWnd, EFUSE_MSG, NULL, NULL);
    EFUSE_ExitEfuse();
    return -1;
}
```

2.14 EFUSE_ReadMacAddr

ÿÿÿint EFUSE_ReadMacAddr(char* mac, int len)

Description: Read MAC address.

Input parameters: mac - array to store MAC addresses

len - the length of the array storing the MAC address, greater than or equal to 6

Return value: status value, 0 - success, non-0 - failure

Example:

```
int i;
char mac[6];
CString strTmp;
pMain->m_strMacAddr = _T("");
if (EFUSE_ReadMacAddr(mac, sizeof(mac)))
{
    pMain->m_strMsg.Format(_T("%s"), EFUSE_GetMsg());
    SendMessage(pMain->m_hWnd, EFUSE_MSG, NULL, NULL);
    EFUSE_ExitEfuse();
    return -1;
}
for (i = 0;i < 6;i++)
{
    strTmp.Format(_T("%02X"), mac[i]);
    pMain->m_strMacAddr += strTmp;
}
```

2.15 EFUSE_WriteUserArea

ÿÿÿint EFUSE_WriteUserArea(char* data, int startAddr, int len)

Description: Write data into the user area starting from the specified bit address.

Input parameters: data - array to store user area data

startAddr - starting bit address, 0~600, unit is bit

len - length of user area data to be written, 1~601, unit is bit

Return value: status value, 0 - success, non-0 - failure

Example:

```
int i;
BYTE uaData[76];
CString strLeft, strRight;
WORD startBitAddr, bitLen, byteLen;
CString strUserAreaData, strStartBitAddr, strBitLen;
strUserAreaData = pMain->m_strUserAreaData0;
strStartBitAddr = pMain->m_strStartBitAddr0;
strBitLen = pMain->m_strBitLen0;
strRight = strUserAreaData;
startBitAddr = (WORD)_tcstoul(strStartBitAddr, NULL, 0);
bitLen = (WORD)_tcstoul(strBitLen, NULL, 0);
byteLen = bitLen / 8;
if (bitLen % 8) byteLen++;
for (i = 0;i < byteLen;i++)
{
        strLeft = strRight.Left(2);
        strRight = strRight.Right(strRight.GetLength() - 2);
        uaData[i] = (BYTE)_tcstoul(strLeft, NULL, 16);
}
if (EFUSE_WriteUserArea((char*)uaData, startBitAddr, bitLen))
{
        pMain->m_strMsg.Format(_T("%s"), EFUSE_GetMsg());
        SendMessage(pMain->m_hWnd, EFUSE_MSG, NULL, NULL);
        EFUSE_ExitEfuse();
        return -1;
}
return 0;
```

2.16 EFUSE_ReadUserArea

Prototype: int EFUSE_ReadUserArea(char* data, int startAddr, int len)

Description: Read the user area data starting from the specified bit address.

Input parameters: data - array to store user area data

startAddr - starting bit address, 0~600, unit is bit

len - length of user area data to be written, 1~601, unit is bit

Return value: status value, 0 - success, non-0 - failure

Example:

```
int i;
BYTE uaData[76];
CString strTmp;
WORD startBitAddr, bitLen, byteLen;
CString strUserAreaData, strStartBitAddr, strBitLen;
strUserAreaData = pMain->m_strUserAreaData0;
strStartBitAddr = pMain->m_strStartBitAddr0;
strBitLen = pMain->m_strBitLen0;
strUserAreaData = _T("");
startBitAddr = (WORD)_tcstoul(strStartBitAddr, NULL, 0);
bitLen = (WORD)_tcstoul(strBitLen, NULL, 0);
byteLen = bitLen / 8;
if (bitLen % 8) byteLen++;
if (EFUSE_ReadUserArea((char*)uaData, startBitAddr, bitLen))
{
        pMain->m_strMsg.Format(_T("%s"), EFUSE_GetMsg());
        SendMessage(pMain->m_hWnd, EFUSE_MSG, NULL, NULL);
        EFUSE_ExitEfuse();
        return -1;
}
for (i = 0;i < byteLen;i++)
{
        strTmp.Format(_T("%02X"), uaData[i]);
        strUserAreaData += strTmp;
}
return 0ÿ
```

2.17 EFUSE_WriteUserArea872

ÿÿÿint EFUSE_WriteUserArea872(char* data, int startAddr, int len)

Description: Write data into the user area starting from the specified bit address.

Input parameters: data - array to store user area data

startAddr - starting bit address, 0~351, unit is bit

len - length of user area data to be written, 1~352, unit is bit

Return value: status value, 0 - success, non-0 - failure

Example: Please refer to the usage example of EFUSE_WriteUserArea() interface.

2.18 EFUSE_ReadUserArea872

Prototype: int EFUSE_ReadUserArea872(char* data, int startAddr, int len)

Description: Read the user area data starting from the specified bit address.

Input parameters: data - array to store user area data

startAddr - starting bit address, 0~351, unit is bit

len - length of user area data to be written, 1~352, unit is bit

Return value: status value, 0 - success, non-0 - failure

Example: Please refer to the usage example of EFUSE_ReadUserArea() interface.

3 Sample Tools

Considering the simplicity and convenience, the sample project is written in MFC and compiled in VS2008.

The sample tool interface is as follows:

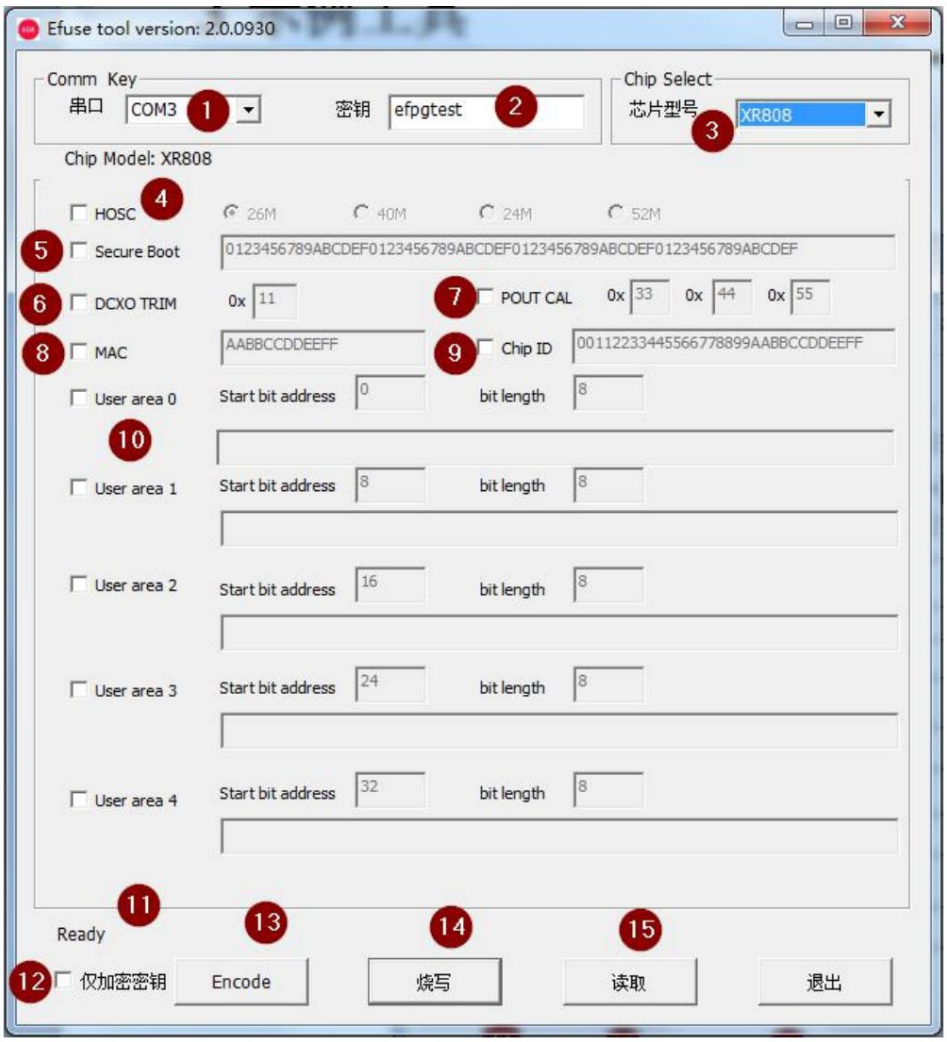

Figure 3-1 Screenshot of the sample tool

1. Serial port selection;

2. Key string;

3. Select the chip;

4. Choose whether to read and write HOSC;

5. Select whether to read and write Secure Boot;

6. Choose whether to read and write DCXO TRIM;

7. Select whether to read and write POUT CAL;

8. Choose whether to read and write MAC addresses;

9. Select whether to read Chip ID;

10. Select whether to read and write the User Area. You can select up to 5 addresses for reading and writing at the same time.

11. Information bar;

12. Encryption test selection, when checked, click the "Encode" button, the key will be encrypted separately, and the result will be displayed in Information bar. When it is not checked, click the "Encode" button, and the encryption result of the last command will be displayed in the information bar;

13.Encode encryption test button;

14. Start the burning function button;

15. Start the function button of reading the burning result;